

ARE 212 SECTION 11: Instrumental Variables

Fiona Burlig

April 7, 2016

Off we go

This week, we'll discuss an incredibly valuable (but probably also overused) tool for the applied econometrician: instrumental variables (IV). We'll talk through what IV actually does, and show a couple of different ways to implement it - the right one to use depends on the scenario you find yourself in.

As you well know by now, OLS ceases to be consistent when our strict exogeneity/population orthogonality condition is violated. As soon as one of our right-hand side variables is correlated with the error term, we're in big trouble. There are three key situations you'll likely run into where this might happen: omitted variable bias, which we've already seen, simultaneous equations models, which you'll talk about in class with Max (but we won't have time to touch in section), and measurement error in your right-hand-side variables (which is really just a special case of omitted variable bias).

We've seen (many times) before that omitted variable bias causes problems in OLS regressions. The good news is that we're finally learning about a tool to fix this kind of bias! To think about that, let's drill down into what omitted variable bias does. We'll start with a very simple case, and then extend this to make it a little more complicated. Suppose your true DGP is the following:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \varepsilon$$

Let's make a couple more assumptions. First, assume that x_1 and x_2 are uncorrelated with ε . Suppose further that x_1 and x_2 are correlated with one another - that is, $\text{Cov}(x_1, x_2) \neq 0$. Finally, suppose that you, the econometrician, don't actually *observe* x_2 .¹

Uh oh. We have a problem: even though we don't actually see x_2 , we really want to be able to estimate β_1 ! The naive non-ARE 212 student might regress y on just x_1 - but you of course know better.² That would look like this:

$$y = \beta_0 + \beta_1 x_1 + \nu$$

where $\nu = \beta_2 x_2 + \varepsilon$. But this means that x_2 is in our error term - and since x_1 and x_2 are correlated, we've got a problem. x_1 has become *endogenous*.³

¹This might sound random and arbitrary, but there are many situations in which we don't actually observe some part of our DGP. A classic example is if you want to run a regression of wages on years of schooling. We of course think that ability is correlated with years of schooling, and also affects wages. But we don't have an ability-o-meter. If you do, quit your day job and go become a college admissions officer/HR rep.

²You'd better.

³Booga booga.

As if by magic...

The good news is that (as always in this class) some smart econometricians before us have figured out how to handle endogenous variables. All we need is an instrumental variable. What's an instrumental variable, you ask? Great question. Very loosely, an IV is a variable that drives/is correlated with the “good” or “exogenous” variation in x_1 , but is unrelated to the “bad” or “endogenous” or “related-to- x_2 ” variation in x_1 . More formally, an IV is a variable (we'll call it z) that satisfies the following two properties:

1. $\text{Cov}(z, x_1) \neq 0$
2. $\text{Cov}(z, \nu) = 0$

The first condition tells us that z and x_1 are correlated - if this weren't true, we'd be in trouble still. Remember that we're trying to get a consistent estimate of $\hat{\beta}_1$. If our instrument is totally unrelated to x_1 , we won't have any hope of using it to try to get at β_1 .⁴ The second condition, commonly called the “exclusion restriction” says that z has to affect y *only* through x_1 . Notice that this also implies that $\text{Cov}(z, \varepsilon) = 0$, because we've already assumed that x_2 is uncorrelated with ε .⁵

It turns out that in real life, coming up with z s that satisfy the first condition is trivial - and the good news is that we can easily test the validity of this assumption - but coming up with z s that satisfy the exclusion restriction is extremely difficult. Because we don't observe ε , we also can't ever test this assumption. This should make you highly skeptical of anybody doing instrumental variables regressions, and wary of trying them yourself. IV is often more art than science, and the quality of any IV paper will hinge directly on your ability to convince people that the exclusion restriction is satisfied. More on this later.

All that said, if you do manage to find a good instrument, it will allow you to actually recover a consistent estimate of $\hat{\beta}_1$, in a situation where we have no chance of estimating it consistently otherwise, so it's a very powerful tool. Given that we've found one of these unicorn instruments, how do we actually use it to get a consistent estimate of $\hat{\beta}_1$? We have a couple of tools at our disposal to do this - let's go through them and code them up in R as we go.

Nursing school

How do we actually run an IV estimate?⁶ The instrumental variables estimator is deceptively simple:

$$\hat{\beta}_{IV} = (Z'X)^{-1}Z'y$$

Z contains all of the “exogenous” variables in your regression, including your intercept and your instrument. In the simple case we've written up above, the only “exogenous” variable other than your instrument itself is the intercept - but you could imagine a slightly more complicated DGP where there's also an x_3 term that's observed and uncorrelated with ε and x_2 . X , then, contains all of your endogenous variables - in our case, just x_1 - *as well as* all of your non-instrument exogenous variables (just our intercept for now). The mechanical reason behind this is that because we're matrix multiplying Z' and X , they have to be the same dimension. Finally, y is just our

⁴If we try to regress number of students at UC Berkeley on price of tuition, and we try to instrument for tuition costs with windspeed over the middle of the Pacific Ocean, we're not going to get very far.

⁵I've written this out slightly differently than you might see it in other places - be wary! Notice that in Max's notes, the second condition is written as $\text{Cov}(z, \varepsilon) = 0$. This is because he assumes that $\text{Cov}(x_2, \varepsilon) \neq 0$. The idea is the same: the exclusion restriction requires that z affects y only through x_1 . If x_2 and ε are correlated, and z and ε are *uncorrelated*, we're good. In the way I've written this up, x_2 and ε are uncorrelated, but x_2 and ν are correlated, so we need z and ν to be uncorrelated.

⁶IV? Get it? This might be one of my worst jokes yet.

normal y . Confused? Let's write this up in R, and (hopefully) all will become clear.

As always, we'll start with our set-up (we'll bring the MASS package back to draw from its multivariate normal distribution):

```
##### PACKAGES
library(MASS)
library(dplyr)
library(ggplot2)
library(lfe)

##### FUNCTIONS
as.tbl_df <- function(data) {
  dataset <- as.data.frame(data) %>%
    tbl_df()
}

tbl_dfGrabber <- function(data, varnames) {
  matrixObject <- data %>%
    select_(.dots = varnames) %>%
    as.matrix()
}

miniOLS <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  return(betahat)
}

bigOLS <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)
  betahat <- miniOLS(data, y, X)
  yhat <- xdata %*% betahat
  e <- ydata - yhat
  s2 <- (t(e) %*% e) / (n-k)
  XpXinv <- solve(t(xdata) %*% xdata)
  se <- sqrt(s2 * diag(XpXinv))
  tStat <- (betahat - 0) / se
  pVal <- tStat %>%
    abs() %>%
    "*"(-1) %>%
    pt(df = n - k) %>%
```

```

"*(2)

# do the F test

# nr of restrictions = nr of X variables (- intercept)
J <- length(X) - 1
# create an identity matrix of size k X k
bigR <- diag(J) %>%
# and attach a column of zeros at the front of it
cbind(0, .)

# intermediate step
RXXinvR <- solve(bigR %*% solve(t(xdata) %*% xdata) %*% t(bigR))

# f statistic
FStat <- t(bigR %*% betahat) %*% RXXinvR %*% (bigR %*% betahat) / (s2 * J)
# p value for the F stat
pValF <- 1 - pf(FStat, df1 = J, df2 = n-k)

olsOut <- list(betahat, se, tStat, pVal, FStat, pValF, yhat)
names(olsOut) <- c("betahat", "SE", "tStat", "tStatPVal", "FStat", "FStatPVal", "yhat")
return(olsOut)
}

##### RANDOMIZATION SEED
set.seed(12345)

##### GGLOT SETUP
myThemeStuff <- theme(panel.background = element_rect(fill = NA),
  panel.border = element_rect(fill = NA, color = "black"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  axis.ticks = element_line(color = "gray5"),
  axis.text = element_text(color = "black", size = 10),
  axis.title = element_text(color = "black", size = 12),
  legend.key = element_blank()
)

```

Okay. To get started, we need to generate some data according to our DGP from above. We are pros at creating data in which x_1 and x_2 are correlated with one another - the only wrinkle here is that we *also* have to generate z , which must be correlated with x_1 but not with x_2 . Good thing we're expert econometricians, and know that all this requires is drawing data from (say) a multivariate normal distribution with the following variance-covariance matrix:

$$\Sigma = \begin{bmatrix} 1 & \rho_{12} & \rho_{1z} \\ \rho_{21} & 1 & 0 \\ \rho_{z1} & 0 & 1 \end{bmatrix}$$

Notice that $\rho_{12} = \rho_{21}$ and $\rho_{z1} = \rho_{1z}$.

Good thing also that we're expert R users, and coding this up is no problem:

```
e <- rnorm(10000)
sigmaMat <- matrix(c(1, 0.75, 0.25, 0.75, 1, 0, 0.25, 0, 1), nrow = 3)
# check to make sure we did it right
sigmaMat

##      [,1] [,2] [,3]
## [1,] 1.00 0.75 0.25
## [2,] 0.75 1.00 0.00
## [3,] 0.25 0.00 1.00

#draw our data
xData <- mvrnorm(n = 10000, mu = c(5, 10, -5), Sigma = sigmaMat)
# impose a DGP
y <- 5 + 2*xData[,1] - 3*xData[,2] + e
z <- xData[,3]
#create our dataset
myData <- cbind(y, 1, xData) %>%
  as.tbl_df()
names(myData) <- c("y", "ones", "x1", "x2", "z")
```

One thing to notice: just figuring out the right Σ matrix, and thinking through how to code it up, was non-trivial. This should already suggest to you that finding a real variable that satisfies the properties you need for an IV is going to be very hard. As we've stressed plenty of times, finding a good IV is more art than science.

Let's make sure that everything worked. We can check the correlations between our variables to do this⁷:

```
cor(myData$x1, myData$x2)

## [1] 0.7473104

cor(myData$x1, myData$z)

## [1] 0.25224

cor(myData$x2, myData$z)

## [1] -0.006175205
```

When we regress y on an intercept, x_1 , and x_2 , we should recover the correct coefficients:

```
miniOLS(myData, "y", c("ones", "x1", "x2"))

##           y
## ones 4.756742
## x1 1.989106
## x2 -2.970279
```

⁷Cough this is the *Pearson correlation coefficient*, cough.

Beautiful. We should also be screwed if we regress y (and an intercept) on just x_1 or just x_2 :

```
miniOLS(myData, "y", c("ones", "x1"))

##              y
## ones -13.8744307
## x1    -0.2219872

miniOLS(myData, "y", c("ones", "x2"))

##              y
## ones -0.2132817
## x2    -1.4779995
```

Less beautiful, but reassuring that we're doing things right.

Let's go ahead and code up the IV estimator - it looks just like the OLS estimator, except instead of $(X'X)^{-1}X'y$, we're calculating $(Z'X)^{-1}Z'y$. We'll make our function take 5 inputs - a dataset, the name of our y variable, the name(s) of our exogenous (non-instrument) variables, the name of our endogenous variable, and the name of our instrument. Note that we don't input the omitted variable - if we had to do that, we'd be back to square one:

```
ivEst <- function(data, y, exog, endog, instr) {
  ydata <- tbl_dfGrabber(data, y)
  exogdata <- tbl_dfGrabber(data, exog)
  endogdata <- tbl_dfGrabber(data, endog)
  instrdata <- tbl_dfGrabber(data, instr)

  xmat <- cbind(endogdata, exogdata)
  zmat <- cbind(instrdata, exogdata)

  betahat <- solve(t(zmat) %*% xmat) %*% t(zmat) %*% ydata
  rownames(betahat) <- c(endog, exog)

  return(betahat)
}
```

Let's test it out:

```
myIV <- ivEst(myData, "y", "ones", "x1", "z")
myIV

##              y
## x1         2.052925
## ones -25.246730
```

2.05 is pretty darn close to 2. Good! We can also check this against the canned routine. We can implement IV through our old friend `feelm()`:

```
# this syntax is a little unwieldy, but it's correct
# regress y on an intercept, with 0 FEs, and then IV for x1 with z
cannedIV <- felm(data = myData, y ~ 1 | 0 | (x1 ~ z))
cannedIV

## (Intercept)    `x1(fit)`
##      -25.247        2.053
```

These go to 11 (well, 2, actually)

It turns out that in the case of *one* instrument and *one* endogenous regressor, $\hat{\beta}_{IV}$ is exactly equal to another estimator: $\hat{\beta}_{2SLS}$. This is the coefficient estimate from a procedure that turns out to be useful for building intuition as to what IV is actually doing, called two-stage least squares (2SLS).

2SLS, not surprisingly, proceeds in two (least squares regression) stages.⁸ First, we run the “first stage”:⁹

$$x_1 = \gamma_0 + \gamma_1 z + u$$

We then use the estimated $\hat{\gamma}$ coefficients to generate predicted values, $\hat{x}_1$

$$\hat{x}_1 = \hat{\gamma}_0 + \hat{\gamma}_1 z$$

Notice that our new $\hat{x}_1$ is orthogonal to ε by construction, since z is orthogonal to ε . In hand-wavy terms, we’re “partialling out” the “bad variation” in x_1 , and leaving ourselves with only the “good variation.” Next, we run the “second stage”, where we regress:

$$y = \beta_0 + \beta_1 \hat{x}_1 + \varepsilon$$

where we’ll now be able to consistently estimate $\hat{\beta}_1$. Magical! Note that if you have other exogenous regressors, you’d want to include them in *both* the first stage and the second stage, or things will go wonky.

This is also relatively easy to code up. Let’s do it:

```
twoStageLS <- function(data, y, exog, endog, instr) {
  ydata <- tbl_dfGrabber(data, y)
  exogdata <- tbl_dfGrabber(data, exog)
  endogdata <- tbl_dfGrabber(data, endog)
  instrdata <- tbl_dfGrabber(data, instr)

  # set up the first stage regressors
  s1RHS <- cbind(instrdata, exogdata)

  # first stage
  gammahat <- solve(t(s1RHS) %*% s1RHS) %*% t(s1RHS) %*% endogdata
  # save predicted x1 hat values
```

⁸Did I mention that econometricians are super creative with their naming conventions? Yeah. That’s a thing.

⁹See?

```

x1hat <- s1RHS %*% gammahat
# set up the 2nd stage regressors
s2RHS <- cbind(x1hat, exogdata)

# run the second stage
betahat <- solve(t(s2RHS) %*% s2RHS) %*% t(s2RHS) %*% ydata
rownames(betahat) <- c(endog, exog)
return(betahat)
}

```

And run it:

```

myTwoStageLS <- twoStageLS(myData, "y", "ones", "x1", "z")
myTwoStageLS

##              y
## x1      2.052925
## ones -25.246730

```

Amazing. It works - and is exactly equivalent to $\hat{\beta}_{IV}$, just like we postulated above. One warning: people sometimes come up with the bright idea to run a logit, probit, Poisson, or other non-linear regression as the first stage of a 2SLS procedure. The above math should make it clear that this is a very bad idea - it's even sometimes called the “forbidden regression.” Why? Neither the conditional expectation function, nor linear projection, work with non-linear functions. This means that only a linear first stage will get you estimates of $\hat{x}$ that are orthogonal to the residuals. In other words, you can't “project out” the bad variation in x when you use a non-linear first stage, so you won't get consistent $\hat{\beta}$ s, which defeats the whole purpose of doing IV in the first place. Just say no, kids.

Another cool thing to notice requires discussing one final equation: the “reduced form”:

$$y = \pi_0 + \pi_1 z + \eta$$

which is just the regression of y on your instrument (and exogenous variables if you have them). It turns out that yet another way of recovering consistent estimates of $\hat{\beta}_1$ with one endogenous variable and one instrument is to estimate the reduced form, and divide it by the first stage. Check it:

```

twoSLSRF <- function(data, y, exog, endog, instr) {
  ydata <- tbl_dfGrabber(data, y)
  exogdata <- tbl_dfGrabber(data, exog)
  endogdata <- tbl_dfGrabber(data, endog)
  instrdata <- tbl_dfGrabber(data, instr)

  # set up the first stage regressors
  s1RHS <- cbind(instrdata, exogdata)

  # first stage
  gammahat <- solve(t(s1RHS) %*% s1RHS) %*% t(s1RHS) %*% endogdata

  # set up the RF regressors

```



```

s2RHS <- cbind(instrdata, exogdata)

# run the second stage
pihat <- solve(t(s2RHS) %*% s2RHS) %*% t(s2RHS) %*% ydata

betahat <- pihat / gammahat
rownames(betahat) <- c(endog, exog)

return(betahat)
}

```

Test it:

```

myTwoStageLSRF <- twoSLSRF(myData, "y", "ones", "x1", "z")
myTwoStageLSRF

##           y
## x1      2.052925
## ones -1.991572

```

Boom! Again, we're getting the right coefficient on x_1 . Why does this work? The reduced form is (essentially) the effect of z on y , and the first stage is (essentially) the effect of z on x_1 - to get at the effect of x_1 on y , all we need to do is to estimate the effect of z on y , and scale it by the effect of z on x_1 - which is exactly what we're doing here. Note that this ratio trick *won't* work if you have, say, multiple endogenous variables and multiple instruments, or additional exogenous variables.

It turns out that IV is just a special case of 2SLS - if you have multiple endogenous variables and multiple instruments, or multiple exogenous variables (you can think of these variables as instruments for themselves), or even multiple instruments for a single endogenous variable, you can still use 2SLS to get consistent estimates of $\hat{\beta}$. The easiest way to do this is just to write $\hat{\beta}_{2SLS}$ in matrix form, like this:

$$\hat{\beta}_{2SLS} = (X'P_ZX)^{-1}(X'P_Zy)$$

where $P_Z = Z(Z'Z)^{-1}Z'$. All of the matrix algebra, but neatly wrapped up into one step. Coding it up is no problem:

```

twoSLSMat <- function(data, y, exog, endog, instr) {
  ydata <- tblldfGrabber(data, y)
  exogdata <- tblldfGrabber(data, exog)
  endogdata <- tblldfGrabber(data, endog)
  instrdata <- tblldfGrabber(data, instr)

  # set up the matrices

  xmat <- cbind(endogdata, exogdata)
  zmat <- cbind(instrdata, exogdata)

  pzmat <- zmat %*% solve(t(zmat) %*% zmat) %*% t(zmat)
}

```

```

betahat <- solve(t(xmat) %*% pzmat %*% xmat) %*% t(xmat) %*% pzmat %*% ydata
rownames(betahat) <- c(endog, exog)
return(betahat)
}

```

And testing it:

```

myTwoStageLSMat <- twoSLSMat(myData, "y", "ones", "x1", "z")
myTwoStageLSMat

##           y
## x1      2.052925
## ones -25.246730

```

One nice thing to highlight about this whole procedure: if you have one endogenous variable and *more than one* instrument for that endogenous variable, the regular IV estimator won't work. Why? Remember that the IV estimator is:

$$\hat{\beta}_{IV} = (Z'X)^{-1}Z'y$$

where X and Z are $k \times n$, and y is $n \times 1$. What happens if Z is, say, $k+1 \times n$? We'll end up with non-conformable matrices, that's what. But! Remember that the 2SLS estimator is:

$$\hat{\beta}_{2SLS} = (X'P_ZX)^{-1}(X'P_Zy)$$

Since P_Z is $n \times n$, it doesn't matter what size Z is - this will still work. It's also easy to imagine how to run 2SLS (in non-matrix form) with multiple instruments: just include more stuff on the right-hand side of your first stage. Easy-peasy, lemon-squeezy.

One thing to notice: our estimate of β_0 isn't consistent, regardless of the estimator we use. The good thing is that we (almost never) actually care about the estimate of the intercept anyway. The good news is that we can get the other β coefficients right.

Standard errors strike back

I told you that we were done dealing with standard errors - but that was a little white lie. We do still need to think carefully about what our standard errors should look like when we're dealing with instrumental variables. We're going to think through this via 2SLS, since it's the most common way to implement the IV estimator. It turns out that it's easy to show that $\mathbb{V}(\hat{\beta}_{2SLS}) = \sigma^2(X'P_ZX)^{-1} \geq \sigma^2(X'X)^{-1} = \mathbb{V}(\hat{\beta}_{OLS})$.¹⁰ The intuition is also clean: in 2SLS, we're "projecting out" the "bad" variation in our endogenous x variables, which leaves us with less overall variation to work with. Less estimating variation means we'll end up with a higher variance.

So how do we estimate the variance of $\hat{\beta}_{2SLS}$? This is very easy to do: we can just calculate $\hat{\sigma}^2(X'P_ZX)^{-1}$, right? We know what X and P_Z are - just data. So all we need is $\hat{\sigma}^2$. as usual, we can use $\frac{e'e}{n}$ as an estimate of σ^2 . We just have to be careful to use the right residuals. The correct residuals are $e = y - x\hat{\beta}_{2SLS}$. Importantly, these are *not the same* as what you'd get if you calculated $y - \hat{x}\hat{\beta}_{2SLS}$! Let's code this up into our 2SLS estimator:

¹⁰Hint: use the linear algebra fact that $X'WX = X'WWX = X'W'WX = (WX)'(WX) \geq 0$ for any symmetric, idempotent matrix W - and remember that P_Z is a symmetric and idempotent matrix.

```

twoStageLS <- function(data, y, exog, endog, instr) {
  ydata <- tblldfGrabber(data, y)
  n <- nrow(ydata)
  exogdata <- tblldfGrabber(data, exog)
  endogdata <- tblldfGrabber(data, endog)
  instrdata <- tblldfGrabber(data, instr)

  # set up the matrices

  xmat <- cbind(endogdata, exogdata)
  zmat <- cbind(instrdata, exogdata)

  pzmat <- zmat %*% solve(t(zmat) %*% zmat) %*% t(zmat)
  betahat <- solve(t(xmat) %*% pzmat %*% xmat) %*% t(xmat) %*% pzmat %*% ydata
  rownames(betahat) <- c(endog, exog)

  e <- ydata - xmat %*% betahat
  s2 <- as.numeric((t(e) %*% e) / n)
  vcov <- s2 * (solve(t(xmat) %*% pzmat %*% xmat))

  se <- sqrt(diag(vcov))

  output <- list(betahat, se)
  names(output) <- c("betahat", "se")
  return(output)
}

```

And test it:

```

twoStageLSOut <- twoStageLS(myData, "y", "ones", "x1", "z")
twoStageLSOut

## $betahat
##           y
## x1      2.052925
## ones -25.246730
##
## $se
##      x1      ones
## 0.1257574 0.6294437

summary(cannedIV)

##
## Call:
##      felm(formula = y ~ 1 | 0 | (x1 ~ z), data = myData)
##

```

```
## Residuals:
##      Min       1Q   Median       3Q      Max
## -12.8662  -2.0888   0.0024   2.1005  12.2268
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -25.2467      0.6295  -40.11  <2e-16 ***
## `x1(fit)`    2.0529      0.1258   16.32  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.137 on 9998 degrees of freedom
## Multiple R-squared(full model): -1.038   Adjusted R-squared: -1.039
## Multiple R-squared(proj model): -1.038   Adjusted R-squared: -1.039
## F-statistic(full model):266.4 on 1 and 9998 DF, p-value: < 2.2e-16
## F-statistic(proj model): 266.4 on 1 and 9998 DF, p-value: < 2.2e-16
## F-statistic(endog. vars):266.4 on 1 and 9998 DF, p-value: < 2.2e-16
```

Boom! Right on. Notice also that the canned routine is giving us *negative* R^2 values. Turns out that R^2 is basically meaningless for IV estimation. Why?

$$R^2 = 1 - \frac{SSR}{SST} = 1 - \frac{\sum_i (y_i - x_i \hat{\beta}_{OLS})^2}{\sum_i (y_i - \bar{y})^2}$$

As we discussed above, correctly calculating the residuals for 2SLS requires calculating them using x_i , not $\hat{x}_i$, so that When we go to substitute $\hat{\beta}_{2SLS}$ into the above formula, we don't also substitute in $\hat{x}_i$ for x_i . This gives us the following:

$$R^2 = 1 - \frac{SSR}{SST} = 1 - \frac{\sum_i (y_i - x_i \hat{\beta}_{2SLS})^2}{\sum_i (y_i - \bar{y})^2}$$

But this formula should give you the heeby-jeebies. We know that x_i is correlated with our error term! This means that you could be trying to explain y with an artificially high amount of variation. In other words, we no longer have the guarantee that $SSR < SST$ - the 2SLS residuals no longer come from a model that nests the constant-only model. This means that R^2 can be negative, and that we can't use it for F -tests.

Use with caution

I've already mentioned a bunch of times that finding a good IV is often hard. You have to convince people that your exclusion restriction is valid! The best case you can make for this is in an RCT setting: suppose that you want to estimate the effect of (say) getting your household hooked up to electricity on your income. It should be obvious that electrification isn't random! But if you're a clever researcher, you might convince the Kenyan government to work with you to randomly allocate connection subsidies to households. You could just regress income on the (randomly assigned) subsidy, but we don't actually care about the effect of subsidies on income - we care about the effects of electrification on income! In this case, though, provided that your subsidies actually worked, it should be the case that subsidies are correlated with take-up of household connections. Furthermore, because your subsidies were randomly assigned, they satisfy the exclusion restriction. In this setup, you can use

subsidies as an IV for connections, and consistently estimate the causal effect of electricity on income.¹¹ Cool, right?

In most cases, though, you'll have to be more clever than the randomistas, since we most commonly use IV to get around the fact that we don't have a great experiment. You should always be wary of this - any time you run an IV, you'll need to do as much as you can to demonstrate that your exclusion restriction (which, remember, is fundamentally untestable) is valid. Another thing you can do to shore up your case is to show the OLS (non-instrumented) regression alongside the IV regression. If there is in fact bias in your OLS, the OLS and IV estimates will be different. You'll have a much stronger leg to stand on with your IV estimation if there's a solid economic reason as to why the bias is going in the right direction. Even then, though, you'll need to provide justifications for your exclusion restriction. Can I say it one more time? Exclusion. Restriction.

Data aren't perfect?!

It turns out that IV was originally conceived of as a solution to measurement error - and this is a case in which it can be super credibly used. Suppose that you have data from two weather stations, A and B . Each weather station measures the actual temperature outside with error - but, importantly, you can make a case that these biases are independent from one another. In other words, each station itself might measure the true temperature with error, but the error at station A is uncorrelated with the error at station B .¹²

Suppose that station A is closer to our village of interest, so we'd like to use its data - but we know that it measures the real temperature with (classical) error.¹³ We're interested in the relationship between temperature and village income. (Go with me for now that everything else is well-behaved. Suspend your disbelief.)

Suppose the true DGP is the following:

$$inc = \beta_0 + \beta_1 temp + \varepsilon$$

where inc is income and $temp$ is true (unobserved) temperature. Suppose further that station A observes $temp^A = temp + \nu$, where $Cov(\nu, temp) = 0$, $Cov(\varepsilon, temp) = 0$, and $Cov(\nu, \varepsilon) = 0$. In other words, station A 's measurement error is classical. If we just regress inc on $temp^A$, we'll suffer from attenuation bias - bias towards zero.¹⁴ Why do we get bias at all? Notice that we can re-write this to cast it as an omitted variable bias problem:

$$\begin{aligned} y &= \beta_0 + \beta_1(temp^A - \nu) + \varepsilon \\ &= \beta_0 + \beta_1 temp^A - \beta_1 \nu + \varepsilon \\ &= \beta_0 + \beta_1 temp^A + v \end{aligned}$$

where clearly $Cov(\varepsilon, v) \neq 0$.

Thankfully, we're not hosed: we also observe $temp^B = temp + \eta$, where $Cov(\eta, temp) = 0$, $Cov(\eta, \nu) = 0$, and $Cov(\eta, \varepsilon) = 0$. It follows that $Cov(temp^B, \varepsilon) = 0$, and $Cov(temp^A, temp^B) \neq 0$ - these are exactly the conditions we need to use station B 's temperature as an instrument for the temperature at station A . Let's quickly make some fake data and try this out:

¹¹Fun fact: We call the estimated effect of subsidies on income the intent-to-treat estimate, or ITT, and the effect of electricity on income the ATE, or average treatment effect. Actually, this is the *local* average treatment effect (LATE), because you're calculating effects for those people who were moved to get electricity by the subsidies. Interested in learning more? Michael Anderson in 213 will tell you all about this!

¹²Maccini and Yang (AER 2009), does almost exactly this - in a real paper!

¹³This doesn't have to be classical, as long as it's still orthogonal to the error at station B , but let's go with it for now.

¹⁴You should, at some point, do the math to prove this. Follow the bias formulas in Max's notes and this should fall out easily.

```
e <- rnorm(10000)
temp <- rnorm(10000)
## TRUE DGP
inc <- 50 + 27*temp + e

tempA <- temp + rnorm(10000)
tempB <- 3 + temp + rnorm(10000)

regData <- cbind(inc, 1, temp, tempA, tempB) %>%
  as.tbl_df()
names(regData) <- c("inc", "ones", "temp", "tempA", "tempB")
```

Check that we can recover the true DGP using OLS, and that we'll get a biased estimate of β_1 if we just use the data from station A:

```
truth <- miniOLS(regData, "inc", c("ones", "temp"))
truth

##           inc
## ones 50.00876
## temp 26.99933

biased <- miniOLS(regData, "inc", c("ones", "tempA"))
biased

##           inc
## ones 49.86054
## tempA 13.43012
```

IV to the rescue!

```
fixed <- twoStageLS(regData, "inc", "ones", "tempA", "tempB")
fixed

## $betahat
##           inc
## tempA 27.13313
## ones 50.07713
##
## $se
##      tempA      ones
## 0.3931710 0.2708297
```

Pretty darn close. All in all, IV is a powerful tool if used correctly. We'll stop here for now. Next week, we'll head into new territory. We'll spend the next few weeks talking about the following topics, which are useful, but not necessarily directly related to Max's material: power calculations, spatial data, and replication.